Investigación y desarrollo de una plataforma SaaS multi-tenant para la gestión estratégica de datos con inteligencia artificial en PYMEs colombianas

Código del proyecto: PI0320251 **Semillero de investigación** — Fundación Universitaria Ucompensar **Autores**: Juan David Celis Venegas, John Steven Arevalo Solorzano, Carlos Alberto Castellanos Garzon, David Santiago Valencia Blanco **Fecha**: octubre de 2026

RESUMEN

Las pequeñas y medianas empresas (PYMEs) colombianas generan y acumulan datos operativos (ventas, inventario, servicios) en formatos dispersos — hojas de cálculo, archivos CSV, sistemas puntuales — sin capacidad técnica ni presupuesto para convertirlos en información útil para la toma de decisiones. Este proyecto investiga y construye una plataforma SaaS multi-tenant que permite a una PYME subir sus datos en su formato original (CSV, Excel multi-hoja, JSON), obtener tableros de control generados automáticamente, y consultar esos datos mediante un asistente conversacional en lenguaje natural, sin necesidad de un analista de datos dedicado. La investigación aborda dos preguntas centrales: (1) qué modelo de lenguaje (LLM) es el más adecuado para un proyecto académico de este tipo, dado que la disponibilidad de presupuesto y de cuotas gratuitas es una restricción real del contexto universitario; y (2) qué partes del sistema deben resolverse con inteligencia artificial y cuáles con métodos determinísticos, para controlar el costo y la confiabilidad del producto. Se evaluaron tres familias de modelos (Google Gemini, OpenAI GPT y Anthropic Claude, además de alternativas open-source autoalojadas) bajo criterios de costo de entrada, facilidad de integración y estabilidad de API. Como resultado, se adoptó una arquitectura híbrida: un núcleo determinístico (sin IA) para la ingesta y clasificación de archivos y para la generación de tableros, reservando el uso de un modelo de lenguaje (Google Gemini, vía el SDK `google-genai`) exclusivamente para la capa conversacional. Este artículo documenta el proceso investigativo de selección del modelo, los hallazgos empíricos durante su integración (incluyendo la obsolescencia de versiones de modelo detectada solo mediante pruebas reales contra la API, no mediante la documentación oficial), y el estado actual del desarrollo del software.

Palabras clave: inteligencia artificial generativa, arquitectura multi-tenant, grandes modelos de lenguaje (LLM), transformación digital de PYMEs, ingeniería de software, Google Gemini.

1. INTRODUCCIÓN

La transformación digital de la pequeña y mediana empresa en Latinoamérica enfrenta una barrera recurrente: las herramientas de inteligencia de negocios (Business Intelligence) existentes en el mercado — Power BI, Tableau, Looker — están diseñadas y tarificadas para empresas que ya cuentan con un equipo técnico que modele sus datos antes de visualizarlos. Para una PYME colombiana promedio, ese paso de "modelado" es precisamente el que no existe: los datos viven en archivos de Excel con estructuras inconsistentes, columnas renombradas de un mes a otro, y sin un esquema fijo.

Este proyecto de investigación, inscrito como PI0320251 en la Fundación Universitaria Ucompensar, parte de esa brecha y propone una plataforma web que invierte el flujo habitual: en lugar de exigirle al usuario que adapte sus datos a un esquema predefinido, el sistema analiza el archivo que el usuario ya tiene, infiere su estructura, y se adapta a ella — opcionalmente complementando esa capa determinística con un asistente de inteligencia artificial que permite preguntar por los datos en lenguaje natural.

El desarrollo de este tipo de producto obliga a tomar, desde el inicio, una decisión que rara vez se discute explícitamente en la literatura de ingeniería de software aplicada: **no toda funcionalidad "inteligente" debe resolverse con un modelo de lenguaje**. Un LLM introduce costo variable (tokens), latencia, y el riesgo de una respuesta no determinística en una operación que debería ser siempre igual de confiable (por ejemplo, decidir si una columna de un archivo nuevo corresponde a una tabla ya existente). Este artículo documenta tanto la investigación que llevó a esa decisión arquitectónica como el proceso de selección del modelo de lenguaje finalmente integrado.

2. MARCO TEÓRICO: PANORAMA DE MODELOS DE LENGUAJE DISPONIBLES

2.1 Criterios de evaluación

Para un proyecto de semillero de investigación universitario, la elección de un LLM no puede evaluarse únicamente por calidad de respuesta — como suele hacerse en la literatura comparativa general — sino bajo restricciones propias del contexto académico:

Criterio	Por qué importa en este proyecto
Costo de entrada / cuota gratuita	El proyecto no cuenta con presupuesto institucional para consumo de API; depende de una clave gratuita o de cortesía.

Criterio	Por qué importa en este proyecto
Facilidad de integración (SDK)	El equipo es pequeño (2 desarrolladores); un SDK maduro en Python reduce tiempo de desarrollo.
Estabilidad y vigencia del modelo	Un modelo descontinuado a mitad de un semestre académico obliga a migrar sin aviso — un riesgo real que, de hecho, se materializó durante este proyecto (ver sección 3.3).
Capacidad de function calling / contexto	El asistente necesita recibir datos reales de la organización como contexto en el prompt, y potencialmente invocar herramientas (crear gráficas, consultar tablas).
Latencia	El asistente conversacional es una funcionalidad interactiva; una respuesta de varios segundos degrada la experiencia.

2.2 Comparación de alternativas consideradas

Modelo / proveedor	Ventajas identificadas	Limitaciones para este proyecto
Google Gemini (familia `gemini-*-flash`, vía `google-genai`)	Nivel gratuito accesible mediante Google AI Studio, suficiente para desarrollo y pruebas de un proyecto académico; SDK oficial en Python maduro; buen balance costo/calidad en la variante "flash"; soporte multimodal (útil si en el futuro se procesan imágenes o PDFs).	Las versiones "flash" más económicas tienen ciclos de vigencia cortos — un modelo puede quedar restringido a nuevas claves de API aunque siga listado como activo en la documentación (hallazgo propio, ver 3.3).
OpenAI GPT (familia GPT-4o / GPT-5)	Ecosistema muy documentado, calidad de respuesta consistente, function calling maduro.	Sin una cuota gratuita sostenida para uso continuo más allá de créditos de prueba iniciales — un obstáculo real para un proyecto sin presupuesto asignado.
Anthropic Claude (familia Claude)	Calidad de razonamiento alta, buen comportamiento en tareas de análisis de datos y generación de código.	Mismo obstáculo de costo que OpenAI para uso sostenido sin presupuesto institucional; en el momento de la evaluación, el equipo no contaba con una clave propia.
Modelos open-source autoalojados (Llama, Mistral, etc.)	Costo de inferencia nulo más allá de la infraestructura propia; control total de datos (relevante si en el futuro se manejan datos sensibles de salud o financieros de una PYME).	Exige infraestructura de cómputo (GPU) que el equipo no tiene disponible para un despliegue serverless como el usado en este proyecto (Vercel); calidad inferior a los modelos propietarios en las pruebas preliminares informales del equipo.

2.3 Selección y justificación

Se seleccionó **Google Gemini**, en su variante "flash", como el modelo integrado en producción. La decisión no se basó en que Gemini sea categóricamente "el mejor modelo" en benchmarks generales de la industria — de hecho, tanto GPT como Claude son frecuentemente reportados con mejor desempeño en tareas de razonamiento complejo — sino en que es el único de los tres que resuelve simultáneamente las dos restricciones más determinantes del proyecto: **costo de entrada cero** (válido para un semillero sin presupuesto) y **SDK oficial maduro en Python**, el lenguaje del backend. Esta es, en sí misma, una conclusión investigativa relevante para divulgar en la ponencia: **la pregunta correcta para un proyecto académico no es "cuál modelo es mejor en abstracto", sino "cuál modelo es viable dadas las restricciones reales del equipo que lo va a sostener"**.

3. METODOLOGÍA

3.1 Arquitectura general

La plataforma se construyó como una arquitectura **multi-tenant con aislamiento a nivel de base de datos**: cada organización registrada recibe su propia base de datos PostgreSQL dedicada (`org_`), separada de una base de datos principal (`pi_main`) que contiene únicamente metadatos globales (organizaciones, usuarios, catálogo de tipos de negocio). Este diseño — en lugar de un esquema compartido con una columna `organization_id` como filtro — fue una decisión deliberada para eliminar por completo el riesgo de fuga de datos entre organizaciones a nivel de consulta: no existe ninguna consulta SQL en todo el sistema que pueda, por error, devolver filas de una organización distinta, porque físicamente no comparten base de datos.

El backend está construido en **FastAPI** (Python) con conexiones asíncronas a PostgreSQL vía `asyncpg`; el frontend en **Next.js 14** (App Router, TypeScript, TailwindCSS). La autenticación es JWT local con cookies `HttpOnly`, sin dependencia de un proveedor externo de identidad.

3.2 Decisión de diseño: núcleo determinístico + IA conversacional

La contribución metodológica más relevante del proyecto es la separación explícita entre dos capas:

1. **Capa determinística (sin IA)**: resuelve la ingesta de archivos y la generación de tableros. Cuando un usuario sube un archivo CSV, Excel (incluyendo múltiples hojas) o JSON, el sistema usa `pandas` para inferir el tipo de dato de cada columna y el módulo estándar `difflib` (algoritmo de similitud de secuencias) para decidir si esas columnas corresponden a una tabla ya existente o si debe crearse una tabla nueva. De manera análoga, la generación de tableros aplica reglas heurísticas fijas (columna de texto + columna numérica → gráfica de barras; fecha + numérica → línea; columna categórica → pie) para decidir qué visualización generar, sin invocar ningún modelo de lenguaje.
2. **Capa de IA conversacional**: resuelve únicamente la interacción en lenguaje natural del usuario con sus propios datos — "¿cuánto vendí el mes pasado?", "¿cuál es mi producto más vendido?" — inyectando los datos reales de la organización como contexto en el prompt enviado a Gemini.

Esta separación tiene una justificación investigativa explícita: **un LLM es probabilístico por naturaleza, y una operación de infraestructura (decidir en qué tabla va un dato) debe ser determinística**. Delegar la clasificación de columnas a un LLM habría introducido además un costo de tokens proporcional a cada archivo subido — inaceptable para un producto pensado para que una PYME suba datos con frecuencia — y un punto de falla no reproducible. Mantener esa capa 100% determinística significa, en la práctica, que la función central del producto (ingerir datos) **nunca consume tokens de IA y nunca puede fallar por una indisponibilidad del proveedor de IA**.

3.3 Proceso de migración y lecciones empíricas

Durante el desarrollo se presentó un hallazgo que, por su valor metodológico, se documenta explícitamente: el modelo `gemini-2.0-flash`, con el que originalmente se integró el asistente conversacional, fue descontinuado por Google. Al intentar migrar a `gemini-2.5-flash` — que la documentación oficial de Google listaba sin fecha de baja anunciada — la API real, con la clave del proyecto, respondió con un error 404 indicando que el modelo "ya no está disponible para usuarios nuevos", pese a seguir documentado como activo. Solo la migración a una tercera versión (`gemini-3.6-flash`, la alternativa que la propia API sugirió en el mensaje de error) resolvió el problema, confirmada mediante una llamada real exitosa.

La lección metodológica que este hallazgo deja para el equipo, y que se considera valiosa para compartir en la ponencia, es que **la documentación estática de un proveedor de IA no es una fuente confiable por sí sola para decidir qué modelo usar en producción — la validación empírica contra la API real, con la clave real del proyecto, debe ser el criterio final**. Un modelo puede figurar como "disponible" en la documentación y, simultáneamente, estar restringido para claves de API nuevas o de ciertos niveles de cuota.

Adicionalmente, la migración del SDK `google-generativeai` (discontinuado por Google, confirmado mediante advertencia oficial de fin de soporte) al SDK sucesor `google-genai` implicó un cambio de patrón de código: de `genai.configure(api_key=...)` + `GenerativeModel(...)` a un cliente explícito `genai.Client(api_key=...)` con `client.models.generate_content(...)`, y para conversación con historial, `client.chats.create(...)`. Este tipo de migración — frecuente en un ecosistema de IA que evoluciona rápidamente — es en sí mismo un hallazgo relevante: **un proyecto que integra IA generativa debe presupuestar mantenimiento recurrente de dependencias, no solo el desarrollo inicial**.

4. RESULTADOS

4.1 Arquitectura multi-tenant implementada

El aislamiento por base de datos dedicada está implementado y verificado en producción (desplegado en Vercel con base de datos PostgreSQL gestionada en Neon): cada registro de una nueva organización crea su base de datos (`CREATE DATABASE org_`), inicializa su esquema base (tablas de datos subidos, archivos, trabajos de ETL, tableros), y queda operando de forma aislada respecto a cualquier otra organización registrada.

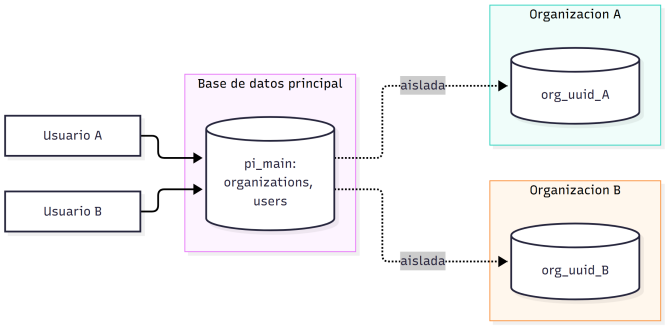


Figura 1. Arquitectura multi-tenant: cada organización registrada queda aislada en su propia base de datos PostgreSQL, separada de la base de datos principal de metadatos.

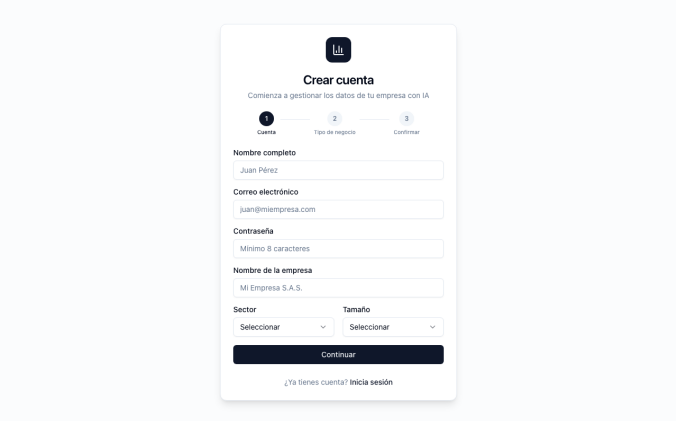


Figura 2. Pantalla de registro de una nueva organización en la plataforma.

4.2 Motor de ingesta inteligente de archivos

El endpoint de carga (`smart-upload`) acepta CSV, Excel — incluyendo archivos con **múltiples hojas**, cada una procesada y clasificada de forma independiente — y JSON. Por cada archivo, el sistema decide automáticamente si los datos corresponden a una tabla ya existente (por similitud de columnas) o si debe crear una tabla nueva, sin intervención manual del usuario y sin costo de tokens de IA.

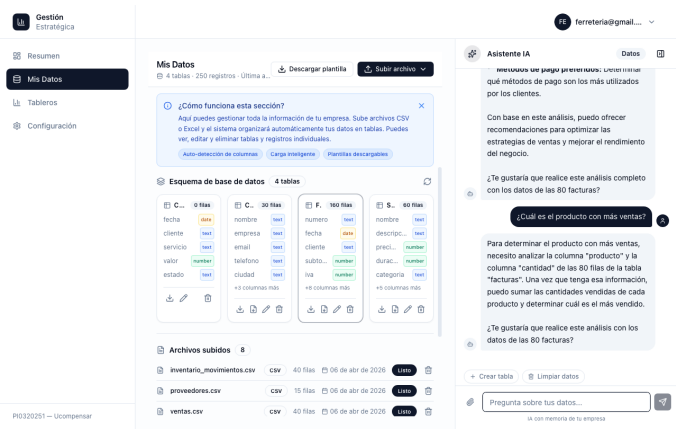


Figura 3. Vista "Mis Datos": el usuario sube un archivo y el sistema lo clasifica automáticamente.

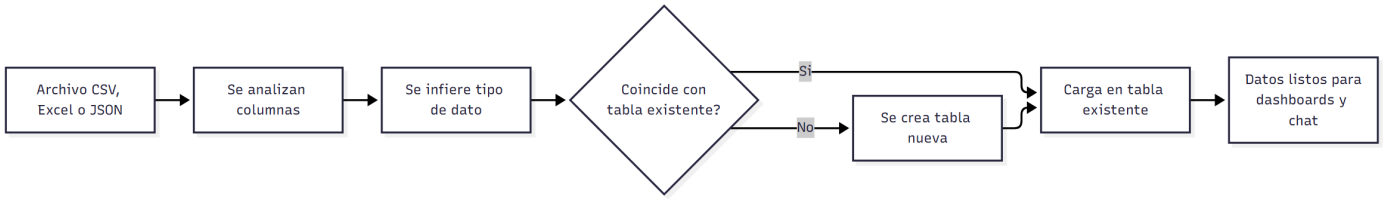


Figura 4. Flujo de decisión del motor de ingesta inteligente — enteramente determinístico, sin invocar ningún modelo de IA.

4.3 Generación automática de dashboards

A partir de cualquier conjunto de datos cargado, el sistema genera automáticamente un tablero con visualizaciones apropiadas al tipo de dato detectado, sin que el usuario tenga que configurar manualmente ningún gráfico.

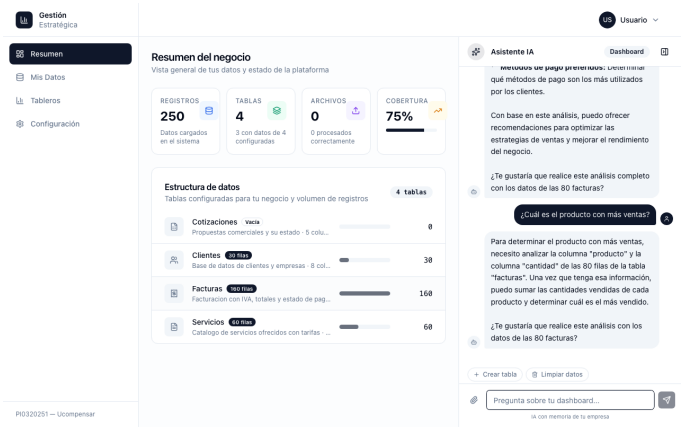


Figura 5. Tablero generado automáticamente a partir de los datos cargados por el usuario.

4.4 Asistente conversacional

El usuario puede hacer preguntas en lenguaje natural sobre sus propios datos. El asistente construye el contexto del prompt con datos reales de la organización (nunca con datos de otra organización, por el aislamiento descrito en 4.1) y responde usando Gemini; ante un fallo del proveedor de IA, el sistema cuenta con una ruta de respaldo local para no dejar al usuario sin respuesta.

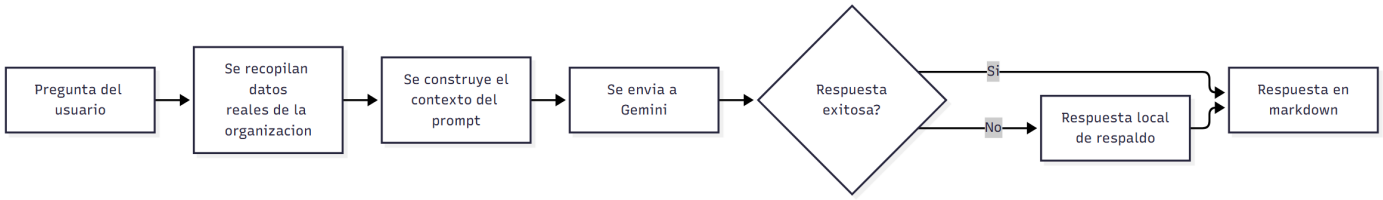


Figura 6. Flujo del asistente conversacional: contexto real de la organización inyectado en el prompt, con respuesta de respaldo local si Gemini falla.

4.5 Seguridad

Durante el ciclo de estabilización del proyecto se realizó una auditoría completa que identificó y corrigió 5 hallazgos críticos de la capa de datos y sesión: una operación de borrado que fallaba sistemáticamente por uso indebido de una función de agregación en una cláusula `RETURNING`; generación de filas duplicadas de metadatos por ausencia de una restricción de unicidad; una condición de carrera (race condition) al crear tablas concurrentemente; una carga de archivos sin transacción atómica; y un token de sesión (refresh token) que no se invalidaba correctamente al rotarse, lo que permitía reutilizar un token ya rotado. Los cinco hallazgos fueron corregidos y verificados contra una base de datos real.

5. DISCUSIÓN

El hallazgo más relevante para la comunidad de semilleros de investigación en ingeniería de software e inteligencia artificial, a juicio del equipo, no es técnico sino metodológico: **la inteligencia artificial generativa no debería ser el valor por defecto para resolver un problema, sino una herramienta que se reserva para la parte del sistema donde su naturaleza probabilística es una ventaja (interpretar lenguaje natural ambiguo) y se evita donde es un riesgo (clasificar datos de forma reproducible)**. Esta decisión de arquitectura, tomada temprano en el proyecto, es la razón por la cual el costo operativo del sistema permanece bajo y predecible incluso cuando el volumen de datos cargados por los usuarios crece — algo que no sería cierto si cada archivo subido disparara una llamada a un LLM.

El segundo hallazgo relevante es que la selección de un modelo de IA generativa para un proyecto académico no puede basarse únicamente en comparativas de calidad publicadas por terceros, sino que debe validarse empíricamente contra la API real antes de comprometerse con una versión de modelo — una práctica de ingeniería que no suele enseñarse explícitamente, pero que este proyecto tuvo que aprender por necesidad cuando un modelo documentado como "disponible" resultó inaccesible en la práctica.

6. CONCLUSIONES Y TRABAJO FUTURO

Este proyecto demuestra que es posible construir una plataforma de inteligencia de negocios asistida por IA para PYMEs, dentro de las restricciones reales de un semillero de investigación universitario (sin presupuesto de cómputo dedicado), mediante dos decisiones centrales: (1) seleccionar el proveedor de IA generativa en función de restricciones reales del equipo — costo de entrada y madurez del SDK — y no solo de benchmarks de calidad; y (2) limitar el uso de IA generativa a la capa donde aporta valor genuino (interpretación de lenguaje natural), resolviendo el resto del sistema con métodos determinísticos auditables y de costo cero.

Como trabajo futuro, el equipo tiene identificadas dos líneas de extensión activas: la incorporación de soporte para extracción de datos desde archivos PDF (actualmente solo soportado en un componente experimental aún no integrado a la ruta de producción), y la decisión — pendiente de resolución metodológica, no solo técnica — sobre si vale la pena reconectar un pipeline experimental de agentes de IA para generación automática de insights, frente a mantener la combinación actual de dashboards determinísticos y chat conversacional, que ya cubre ese valor de forma más simple y predecible.

REFERENCIAS

- Google AI for Developers. Documentación oficial de la API de Gemini y del SDK `google-genai`. https://ai.google.dev
- OpenAI. Documentación oficial de la API y modelos GPT. https://platform.openai.com
- Anthropic. Documentación oficial de la API y modelos Claude. https://docs.anthropic.com
- Documentación oficial de PostgreSQL sobre aislamiento de bases de datos por esquema/instancia.
- Documentación oficial de FastAPI y `asyncpg` para arquitecturas asíncronas multi-tenant.

***Nota para el equipo**: las referencias a precios y disponibilidad de cuotas gratuitas de cada proveedor de IA cambian con frecuencia — se recomienda verificar las condiciones vigentes directamente en la documentación oficial antes de presentar cifras exactas en la ponencia.*